LAPORAN PROYEK PEMOGRAMAN SYSTEM

“Pembuatan Game Battle Of The Port: Memory Siege”



Disusun oleh:

Iren Damai Yanti Sitinjak – 43324026

DIII – TEKNOLOGI KOMPUTER
FAKULTAS VOKASI
INSTITUT TEKNOLOGI DEL
2026

1. Dasar Teori

Konsep yang digunakan pada system operasi yang akan mengimplementasikan dalam game "*Battle of the Ports: Memory Siege"*. Berikut merupakan cakupan yang diimplementasikan pada proyek ini:

1. Kontrol Proses
- Definisi proses sebagai program yang sedang berjalan.
- Proses identifikasi dengan PID menggunakan getpid().
- Konsep fork dan exec untuk membuat proses baru (Player 1 & Player 2 berjalan sebagai proses terpisah).

2. Sinyal
- Definisi signal sebagai notifikasi antar proses.
- Implementasi SIGUSR1 untuk memberi tahu giliran antar pemain dengan kill().
- Pengendali sinyal (sig_hdl()) yang mengubah variabel my_turn saat belokan tiba.

3. Komunikasi Antar Proses (IPC)
- Definisi IPC sebagai mekanisme pertukaran data antar proses.
- Memori bersama untuk menyimpan permainan keadaan (papan, skor, putaran, PID).
- Semaphore untuk sinkronisasi akses shared memory agar tidak terjadi kondisi race .

4. Sinkronisasi Thread
- Definisi thread sebagai alur kontrol dalam proses.
- Perbandingan Semaphore vs Mutex (dalam game, semaphore bernilai 1 digunakan seperti mutex).

5. POSIX Thread (Pthread)
- API standar untuk manajemen thread (pthread_create(), pthread_join(), pthread_mutex_lock()).
- Relevan meskipun game fokus pada multi-proses, karena prinsip sinkronisasi sama dengan mutex/semaphore.

6. Arsitektur Game
- Struktur data GameBoard dalam memori bersama (board, skor, putaran, PID).
- Mekanisme permainan: papan 7x7, 6 tanda ? (3 kapal, 3 zonk), tebakan menghasilkan 💥, ❌, atau #.
- Sinkronisasi: semaphore untuk lock/unlock, signal SIGUSR1 untuk turn, variabel current_turn untuk status aktif.

2. Tujuan Proyek

Tujuan dari pembuatan proyek ini adalah untuk memberikan implementasi nyata terhadap konsep-konsep sistem operasi, khususnya Inter-Process Communication (IPC) pada lingkungan Linux/Unix. Melalui game ini, mahasiswa atau pengguna dapat memahami secara praktis bagaimana teori sistem operasi diterapkan dalam bentuk simulasi interaktif. Secara khusus, proyek ini bertujuan untuk:

- Menerapkan Process Control dengan menjalankan setiap pemain sebagai proses terpisah yang memiliki identitas PID unik.

- Memahami penggunaan Signal sebagai mekanisme notifikasi antar proses untuk mengatur giliran bermain.
- Mengimplementasikan Shared Memory sebagai media penyimpanan bersama yang digunakan kedua pemain untuk berbagi state game.
- Menggunakan Semaphore untuk sinkronisasi akses shared memory dan mencegah terjadinya race condition.
- Mengenalkan Thread Synchronization dan prinsip POSIX Thread sebagai bagian dari pemahaman sinkronisasi resource.
- Membangun Arsitektur Game yang merepresentasikan integrasi teori sistem operasi ke dalam mekanisme permainan berbasis multi-process.

3. Struktur Data Memori Bersama (struct)

Dalam implementasi game ini, seluruh state permainan disimpan di dalam Shared Memory agar dapat diakses oleh kedua proses pemain. Struktur data yang digunakan berbentuk struct dalam bahasa C, yang mendefinisikan papan permainan, status giliran, skor, serta identitas proses pemain.

A. Struktur papan pemain

Pada struktur penyimpanan kondisi papan masing masing pemain:

```
typedef struct {
    char board[BOARD_SIZE][BOARD_SIZE];
    int revealed[BOARD_SIZE][BOARD_SIZE];
    int ship_in_question[BOARD_SIZE][BOARD_SIZE];
} PlayerBoard;
```

- board → menampilkan isi papan (simbol kapal, zonk, atau kosong).
- revealed → menandai apakah suatu kotak sudah ditebak.
- ship_in_question → menyimpan informasi apakah tanda ? berisi kapal atau zonk.

B. Struktur gameboart

Struktur utama yang menyimpan keseluruhan state game:

```
      typedef struct {
          PlayerBoard player1, player2;
          int current_turn;
          int winner;
          int move_count;
          int game_active;
          int player1_score, player2_score;
          pid_t player1_pid, player2_pid;
} GameBoard;
```

- player1, player2 → papan permainan masing-masing pemain.
- current_turn → menentukan giliran pemain yang sedang aktif.
- winner → menyimpan status pemenang.
- move_count → menghitung jumlah langkah yang sudah dilakukan.
- game_active → menandakan apakah game masih berjalan.
- player1_score, player2_score → skor masing-masing pemain.
- player1_pid, player2_pid → identitas proses pemain untuk komunikasi via signal

4. Kode Utama Pemograman

Pada pengkodean pemograman utama, isi yang perlu dituliskan adalah bagian inti dari implementasi game Battle of the Ports: Memory Siege. Sub bab ini berfungsi untuk menunjukkan bagaimana teori yang sudah dibahas (Process Control, Signal, IPC, Semaphore, dll.) benar-benar diterapkan dalam bentuk kode. Berikut poin-poin yang sebaiknya dimasukkan:

- Inisialisasi Shared Memory
  - Kode untuk membuat dan melampirkan shared memory (shmget(), shmat()).
  - Struktur GameBoard dimasukkan ke dalam shared memory.

```
int shmid = shmget(IPC_PRIVATE, sizeof(GameBoard), IPC_CREAT | 0666);
GameBoard *G = (GameBoard *) shmat(shmid, NULL, 0);
```

- Inisialisasi Semaphore

  - Kode untuk membuat semaphore (`semget()`) dan operasi dasar (`semop()`).
  - Digunakan untuk mengunci akses shared memory agar tidak terjadi race condition.

```
// Membuat semaphore
int semid = semget(IPC_PRIVATE, 1, IPC_CREAT | 0666);
// Operasi lock/unlock
void sem_lock(int semid) {
    struct sembuf sb = {0, -1, 0};
    semop(semid, &sb, 1);
}
void sem_unlock(int semid) {
    struct sembuf sb = {0, 1, 0};
    semop(semid, &sb, 1);
}
```

- Process control
  - Kode pemanggilan fork() untuk membuat proses pemain.
  - Menampilkan PID masing-masing pemain dengan getpid().

```
pid_t pid = fork();
if (pid == 0) {
    // Child process → Player 2
    printf("Status: PLAYER 2 ACTIVE (PID: %d)\n", getpid());
} else {
    // Parent process → Player 1
    printf("Status: PLAYER 1 ACTIVE (PID: %d)\n", getpid());
}
```

- Signal Handiling
  - Kode pengiriman signal antar pemain (kill(opp_pid, SIGUSR1)).
  - Implementasi fungsi handler (sig_hdl()) untuk mengatur giliran bermain.

```
// Handler giliran
void sig_hdl(int sig) {
    if (sig == SIGUSR1) {
```

```
            G->current_turn = 1; // Giliran berpindah
        }
    }
    // Registrasi handler
    signal(SIGUSR1, sig_hdl);
    // Kirim signal ke lawan
    kill(G->player2_pid, SIGUSR1);
```

- Loop Permainan

Kode utama yang mengatur jalannya permainan:

- Menampilkan papan.
- Input tebakan koordinat.
- Update skor dan status giliran.
- Mengecek kondisi kemenangan.

```
    while (G->game_active) {
        if (G->current_turn == 0) {
            // Player 1 menebak
            play_turn(&G->player1, &G->player2);
            G->current_turn = 1;
            kill(G->player2_pid, SIGUSR1);
        } else {
            // Player 2 menebak
            play_turn(&G->player2, &G->player1);
            G->current_turn = 0;
            kill(G->player1_pid, SIGUSR1);
        }
    }
```

- Sinkronisasi Game
  - Kombinasi semaphore (lock/unlock) dengan signal untuk memastikan giliran berjalan bergantian.
  - Variabel current_turn dalam shared memory sebagai penentu giliran aktif.

- Terminasi dan Pembersihan
  - Kode untuk melepas shared memory (shmdt()) dan menghapusnya (shmctl()).
  - Menghapus semaphore setelah permainan selesai.

```
    // Melepas shared memory
    shmdt(G);
    shmctl(shmid, IPC_RMID, NULL);

    // Menghapus semaphore
    semctl(semid, 0, IPC_RMID);
```

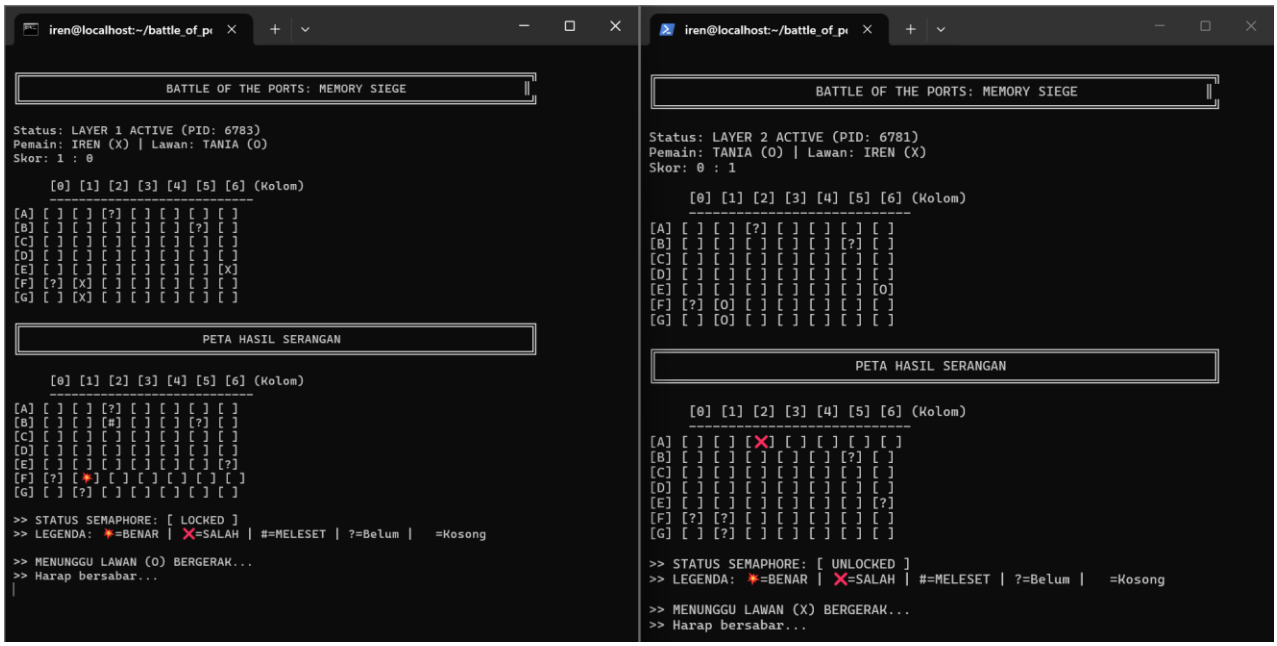
5. Aturan Main Game

Aturan permainan Battle of the Ports: Memory Siege dirancang untuk mensimulasikan konsep Inter-Process Communication (IPC) melalui interaksi antar pemain. Permainan ini menggunakan papan berukuran 7x7 dengan baris A–G dan kolom 0–6.

1. **Papan Permainan**
   - Setiap pemain memiliki papan sendiri dengan 6 tanda ?.
   - Dari 6 tanda tersebut, **3 berisi kapal** dan **3 lainnya zonk (kosong)**.
   - Posisi tanda ? ditentukan secara acak.

2. **Giliran Bermain**
   - Pemain bergiliran menebak koordinat papan lawan.
   - Giliran dikendalikan oleh **signal SIGUSR1** dan variabel current_turn dalam shared memory.
   - Semaphore digunakan untuk memastikan hanya satu pemain yang dapat mengakses shared memory pada satu waktu.
3. **Hasil Tebakan**
   - Jika tebakan mengenai tanda ? berisi kapal → muncul simbol 💥 dan pemain mendapat **+1 poin**.
   - Jika tebakan mengenai tanda ? zonk → muncul simbol ❌ dan giliran berpindah.
   - Jika tebakan mengenai area kosong (tanpa ?) → muncul simbol # dan giliran berpindah.
4. **Kondisi Kemenangan**
   - Pemenang adalah pemain yang pertama kali berhasil menemukan **3 kapal lawan**.
   - Jika kedua pemain mencapai jumlah kapal yang sama pada akhir permainan, maka hasilnya **seri**.
5. **Sinkronisasi Permainan**
   - **Semaphore** digunakan untuk mengunci akses shared memory.
   - **Signal** digunakan untuk memberi tahu lawan bahwa giliran telah berakhir.
   - Status giliran ditampilkan di layar agar pemain mengetahui siapa yang sedang aktif.

5.1 Hasil Eksekusi Dua Proses Paralel



Gambar di atas menunjukkan hasil nyata dari eksekusi game *Battle of the Ports: Memory Siege* ketika dijalankan dengan dua proses paralel pada sistem Linux. Setiap pemain dijalankan sebagai proses terpisah dengan PID unik, ditampilkan pada layar status:

- **Player 1 (IREN/X)** → PID: 6783
- **Player 2 (TANIA/O)** → PID: 6781

Penjelasan Tampilan:

1. **Status Proses**
- Setiap terminal menampilkan identitas pemain, lawan, skor, dan PID proses aktif.
- Hal ini membuktikan bahwa kedua pemain berjalan sebagai proses independen.
2. **Papan Permainan**
- Grid berukuran 7x7 dengan baris A–G dan kolom 0–6.
- Simbol ? menandakan posisi yang belum diketahui (kapal atau zonk).
- Simbol X, *, dan # menunjukkan hasil tebakan (salah, benar, meleset).

3. **Peta Hasil Serangan**
- Menampilkan riwayat tebakan pemain terhadap papan lawan.
- Simbol digunakan sebagai legenda:
    - * → tebakan benar (kapal kena).
    - X → tebakan salah.
    - # → meleset (area kosong).
    - ? → belum ditebak.

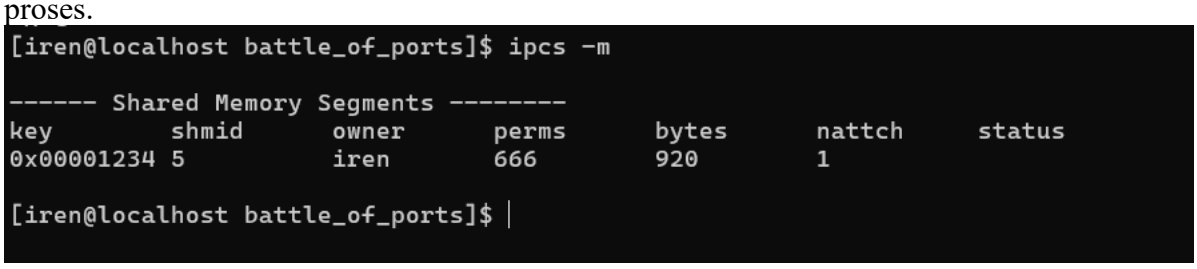
4. **Sinkronisasi Paralel**

- Status semaphore ditampilkan: LOCKED pada Player 1 (sedang giliran), UNLOCKED pada Player 2 (menunggu giliran).
- Signal SIGUSR1 digunakan untuk memberi tahu lawan bahwa giliran telah selesai.
- Variabel current_turn dalam shared memory memastikan giliran bergantian secara konsisten.

5. **Skor dan Progres Permainan**
- Skor ditampilkan secara real-time (IREN unggul 1:0).
- Permainan berlanjut hingga salah satu pemain berhasil menemukan 3 kapal lawan.

5.2 Pemeriksaan Memori Sistem (ipcs -m)

Pemeriksaan memori sistem dilakukan untuk memastikan bahwa game *Battle of the Ports: Memory Siege* benar-benar menggunakan shared memory sebagai media komunikasi antar proses.



Penjelasan pada gambar sebagai berikut:

- Key (0x00001234) → identitas unik shared memory yang dibuat oleh game.
- Shmid (5) → ID shared memory yang digunakan oleh proses untuk operasi shmat(), shmdt(), dan shmctl().
- Owner (iren) → menunjukkan bahwa shared memory dibuat oleh user bernama *iren*.
- Perms (666) → hak akses penuh (read/write) untuk semua user.

- Bytes (920) → ukuran memori yang dialokasikan, sesuai dengan struktur data *GameBoard* yang digunakan dalam game.
- Nattch (1) → jumlah proses yang sedang menempel (attached) ke shared memory saat pemeriksaan dilakukan.

Interpretasi :

- Adanya entri shared memory pada tabel membuktikan bahwa game berhasil mengalokasikan memori bersama untuk menyimpan state permainan.
- Nilai nattch akan berubah sesuai jumlah proses pemain yang sedang aktif.
- Setelah permainan selesai, shared memory harus dilepas dengan shmdt() dan dihapus dengan shmctl(IPC_RMID), sehingga tidak lagi muncul pada daftar ipcs -m.

Kode program:

Common.h

```
  GNU nano 8.1                                          common.h
#ifndef COMMON_H
#define COMMON_H

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#include <sys/sem.h>
#include <signal.h>
#include <time.h>
#include <ctype.h>

#define BOARD_SIZE 7
#define SHM_KEY 0x1234
#define SEM_KEY 0x5678
#define NUM_QUESTION 6
#define NUM_SHIPS 3

typedef struct {
    char board[BOARD_SIZE][BOARD_SIZE];
    int revealed[BOARD_SIZE][BOARD_SIZE];
    int ship_in_question[BOARD_SIZE][BOARD_SIZE];
} PlayerBoard;

typedef struct {
    PlayerBoard player1, player2;
    int current_turn, winner, move_count, game_active;
    int player1_score, player2_score;
    pid_t player1_pid, player2_pid;
} GameBoard;

void sem_lock(int sem);
void sem_unlock(int sem);
void sig_hdl(int sig);
int to_row(char c);

#endif
```

shm_helper.h

```
  GNU nano 8.1                          shm_helper.h
#ifndef SHM_HELPER_H
#define SHM_HELPER_H

#include "common.h"

int get_shm() {
    int shmid = shmget(SHM_KEY, sizeof(GameBoard), 0666 | IPC_CREAT);
    if (shmid == -1) { perror("shmget"); exit(1); }
    return shmid;
}

GameBoard* attach_shm(int shmid) {
    GameBoard *g = (GameBoard*) shmat(shmid, NULL, 0);
    if (g == (GameBoard*) -1) { perror("shmat"); exit(1); }
    return g;
}

void detach_shm(GameBoard *g) { shmdt(g); }
void destroy_shm(int shmid) { shmctl(shmid, IPC_RMID, NULL); }

int get_sem() {
    int semid = semget(SEM_KEY, 1, 0666 | IPC_CREAT);
    if (semid == -1) { perror("semget"); exit(1); }
    semctl(semid, 0, SETVAL, 1);
    return semid;
}

void sem_lock(int sem) { struct sembuf op = {0, -1, 0}; semop(sem, &op, 1); }
void sem_unlock(int sem) { struct sembuf op = {0, 1, 0}; semop(sem, &op, 1); }
void destroy_sem(int semid) { semctl(semid, 0, IPC_RMID); }

#endif
```

player.c

```
  GNU nano 8.1                                                player.c
#include "common.h"
#include "shm_helper.h"

GameBoard *G; int shmid, semid, my_num, my_turn = 0;
pid_t opp_pid;
char my_name[10], opp_name[10];

int to_row(char c) { c = toupper(c); return (c >= 'A' && c <= 'G') ? c - 'A' : -1; }
void sig_hdl(int sig) { if (sig == SIGUSR1) my_turn = 1; }

void init_board(PlayerBoard *p) {
    memset(p, 0, sizeof(PlayerBoard));
    int pos[49][2], total = 0;
    for (int i = 0; i < BOARD_SIZE; i++)
        for (int j = 0; j < BOARD_SIZE; j++)
            { pos[total][0] = i; pos[total][1] = j; total++; }
    srand(time(NULL) ^ (getpid() << 16));
    for (int i = total - 1; i > 0; i--) {
        int j = rand() % (i + 1);
        int t0 = pos[i][0], t1 = pos[i][1];
        pos[i][0] = pos[j][0]; pos[i][1] = pos[j][1];
        pos[j][0] = t0; pos[j][1] = t1;
    }
    for (int i = 0; i < NUM_QUESTION; i++) {
        int r = pos[i][0], c = pos[i][1];
        p->board[r][c] = '?';
        if (i < NUM_SHIPS) p->ship_in_question[r][c] = 1;
    }
}

void print_board() {
    PlayerBoard *p = (my_num == 0) ? &G->player1 : &G->player2;
    PlayerBoard *e = (my_num == 0) ? &G->player2 : &G->player1;

    printf("\n");
    printf("                                                                    \n");
    printf("                    BATTLE OF THE PORTS: MEMORY SIEGE               \n");
    printf("                                                                    \n");
    printf("\n");
    printf("Status: LAYER %d ACTIVE (PID: %d)\n", my_num+1, getpid());
    printf("Pemain: %s (%c) | Lawan: %s (%c)\n",
           my_name, (my_num==0)?'X':'O', opp_name, (my_num==0)?'O':'X');
    printf("Skor: %d : %d\n",
           (my_num==0)?G->player1_score:G->player2_score,
           (my_num==0)?G->player2_score:G->player1_score);
    printf("\n");

    printf("     ");
    for (int j = 0; j < BOARD_SIZE; j++) printf("[%d] ", j);
    printf("(Kolom)\n");
    printf("     ");
    for (int j = 0; j < BOARD_SIZE; j++) printf("-----");
    printf("\n");

    for (int i = 0; i < BOARD_SIZE; i++) {
        printf("[%c] ", 'A' + i);
        for (int j = 0; j < BOARD_SIZE; j++) {
            if (p->board[i][j] == '?') {
                if (p->ship_in_question[i][j]) {
                    printf("[%c] ", (my_num==0)?'X':'O');
                } else {
                    printf("[?] ");
                }
            } else {
                printf("[ ] ");
            }
        }
        printf("\n");
    }
```

```
GNU nano 8.1                                                player.c
    printf("\n");
    printf("                                                    \n");
    printf("                    PETA HASIL SERANGAN             \n");
    printf("                                                    \n");
    printf("\n");
    printf("   ");
    for (int j = 0; j < BOARD_SIZE; j++) printf("[%d] ", j);
    printf("(Kolom)\n");
    printf("   ");
    for (int j = 0; j < BOARD_SIZE; j++) printf("----");
    printf("\n");

    for (int i = 0; i < BOARD_SIZE; i++) {
        printf("[%c] ", 'A' + i);
        for (int j = 0; j < BOARD_SIZE; j++) {
            if (e->revealed[i][j]) {
                if (e->board[i][j] == '?' && e->ship_in_question[i][j]) {
                    printf("[✳] ");
                } else if (e->board[i][j] == '?' && !e->ship_in_question[i][j]) {
                    printf("[❌] ");
                } else {
                    printf("[#] ");
                }
            } else {
                if (e->board[i][j] == '?') {
                    printf("[?] ");
                } else {
                    printf("[ ] ");
                }
            }
        }
        printf("\n");
    }
    printf("\n");
    printf(">> STATUS SEMAPHORE: [ %s ]\n", (G->current_turn == my_num) ? "UNLOCKED" : "LOCKED");
    printf(">> LEGENDA: ✳=BENAR | ❌=SALAH | #=MELESET | ?=Belum |   =Kosong\n");
}

void update_score() {
    G->player1_score = G->player2_score = 0;
    for (int i = 0; i < BOARD_SIZE; i++) for (int j = 0; j < BOARD_SIZE; j++) {
        if (G->player2.revealed[i][j] && G->player2.board[i][j] == '?' && G->player2.ship_in_question[i][j]) G->player1_score++;
        if (G->player1.revealed[i][j] && G->player1.board[i][j] == '?' && G->player1.ship_in_question[i][j]) G->player2_score++;
    }
}

int guess(int row, int col) {
    PlayerBoard *e = (my_num == 0) ? &G->player2 : &G->player1;
    if (row < 0 || row >= BOARD_SIZE || col < 0 || col >= BOARD_SIZE) { return -1; }
    if (e->revealed[row][col]) { return -1; }
    e->revealed[row][col] = 1;
    if (e->board[row][col] == '?') {
        if (e->ship_in_question[row][col]) { return 2; }
        else { return 1; }
    }
    return 0;
}

int check_win() {
    if (G->player1_score >= 3) return 0;
    if (G->player2_score >= 3) return 1;
    if (G->move_count >= BOARD_SIZE * BOARD_SIZE) return 2;
    return -1;
}

int main(int argc, char *argv[]) {
    if (argc != 2 || (argv[1][0] != '1' && argv[1][0] != '2')) {
        printf("                                                    \n");
        printf("                BATTLE OF THE PORTS: MEMORY SIEGE   \n");
        printf("                                                    \n");
        printf("\nUsage: %s [1|2]\n", argv[0]);
        printf("   1 = Player 1 (IREN - X)\n");
        printf("\n ATURAN:\n");
        printf("  • Board 7x7 | Baris A-G | Kolom 0-6\n");
        printf("  • 6 tanda '?' per pemain\n");
        printf("  • 3 berisi KAPAL | 3 ZONK\n");
        printf("  • ✳=BENAR | ❌=SALAH | #=MELESET\n");
        printf("  • Temukan 3 KAPAL lawan untuk MENANG!\n");
        return 1;
    }

    my_num = (argv[1][0] == '1') ? 0 : 1;
    strcpy(my_name, (my_num == 0) ? "IREN" : "TANIA");
    strcpy(opp_name, (my_num == 0) ? "TANIA" : "IREN");

    printf("\n>> Anda adalah PLAYER %d (%s)\n", my_num+1, my_name);

    signal(SIGUSR1, sig_hdl);
    shmid = get_shm(); G = attach_shm(shmid); semid = get_sem();

    if (my_num == 0) {
        sem_lock(semid);
        if (!G->game_active) {
            printf("\n>> Memulai game baru...\n");
            init_board(&G->player1); init_board(&G->player2);
            G->current_turn = G->move_count = 0;
            G->winner = -1; G->game_active = 1;
            G->player1_score = G->player2_score = 0;
            G->player1_pid = getpid(); G->player2_pid = 0;
            my_turn = 1;
            printf(">> Game siap! Anda mulai duluan.\n");
        }
        sem_unlock(semid);
    } else {
        sem_lock(semid);
        while (G->player1_pid == 0) { sem_unlock(semid); usleep(100000); sem_lock(semid); }
        G->player2_pid = getpid();
        sem_unlock(semid);
        my_turn = 0;
        printf("\n>> Bergabung ke game! Menunggu giliran...\n");
    }

    opp_pid = (my_num == 0) ? G->player2_pid : G->player1_pid;
    while (opp_pid == 0) { usleep(100000); opp_pid = (my_num == 0) ? G->player2_pid : G->player1_pid; }

    printf("\n>> Menunggu pemain lain siap...\n"); sleep(1);

    while (G->game_active && G->winner == -1) {
        sem_lock(semid); update_score(); G->winner = check_win(); sem_unlock(semid);
        system("clear");
        print_board();

        if (G->current_turn == my_num && my_turn) {
            char row_char; int col;
            printf("\n>> GILIRAN ANDA (%c)!\n", (my_num==0)?'X':'O');
            printf(">> Masukkan Koordinat (Baris Kolom) - Contoh: C 3\n");
            printf(">> ");
            scanf(" %c %d", &row_char, &col);

            int row = to_row(row_char);

            sem_lock(semid);
            int r = guess(row, col);
            if (r >= 0) {
                system("clear");
                print_board();

                if (r == 2) {
                    printf("\n>> 💣💣💣 BOOM! BENAR! KENA KAPAL! +1 POIN! 💣💣💣\n");
                } else if (r == 1) {
                    printf("\n>> ❌ SALAH! ZONK! Tidak ada kapal...\n");
                } else {
                    printf("\n>> 🌊 MELESET! Kosong...\n");
```



```
                sem_unlock(semid);
                kill(opp_pid, SIGUSR1);
                printf("\n>> Tekan ENTER untuk melanjutkan...");
                getchar(); getchar();
            } else {
                sem_unlock(semid);
                if (row < 0 || row >= BOARD_SIZE || col < 0 || col >= BOARD_SIZE) {
                    printf("\n>> ERROR: Koordinat tidak valid! (Baris A-G, Kolom 0-6)\n");
                } else {
                    printf("\n>> ERROR: Area sudah diserang!\n");
                }
                printf("\n>> Tekan ENTER untuk coba lagi...");
                getchar(); getchar();
            }
        } else {
            printf("\n>> MENUNGGU LAWAN (%c) BERGERAK...\n", (my_num==0)?'O':'X');
            printf(">> Harap bersabar...\n");
            if (!my_turn) pause();
            sem_lock(semid);
            if (G->current_turn == my_num) my_turn = 1;
            sem_unlock(semid);
        }
    }

    system("clear");
    print_board();

    printf("\n");
    printf("╔═══════════════════════════════════════════════════════════╗\n");
    if ((G->winner == 0 && my_num == 0) || (G->winner == 1 && my_num == 1))
        printf("║                  🏆 SELAMAT! ANDA MENANG! 🏆              ║\n");
    else if (G->winner != 2 && G->winner != -1)
        printf("║                     💀 ANDA KALAH! 💀                     ║\n");
    else
        printf("║                     🤝 HASIL SERI! 🤝                     ║\n");
    printf("╚═══════════════════════════════════════════════════════════╝\n");
    printf("\n");
    printf("📊 SKOR AKHIR:\n");
    printf("   IREN (X) : %d     TANIA (O) : %d\n", G->player1_score, G->player2_score);
    printf("   Total tebakan: %d\n", G->move_count);

    detach_shm(G);
    if (my_num == 0) { destroy_shm(shmid); destroy_sem(semid); }
    printf("\n>> Tekan ENTER untuk keluar...");
    getchar(); getchar();
    return 0;
}
```